ROS 2 + PX4 SITL Project Report

Karta Innovations — Candidate Assessment
Hardik Garg

1. Overview

The goal of this project was to build a small but complete robotics pipeline around PX4 SITL, getting a simulated drone flying, reading its state into ROS 2, pulling a camera feed from the simulation, running a basic perception step on it, and recording everything into a rosbag. The emphasis was on getting each piece working and verified individually rather than rushing to a polished demo.
This was done on a fresh Ubuntu 22.04 dual-boot setup with an RTX 3050 GPU, starting from scratch with no prior ROS 2 or PX4 environment on the machine.

2. Approach

Integration Path

The assignment offered two paths: uXRCE-DDS (the recommended, newer approach) or MAVROS (an older but widely used alternative). I have prior experience with MAVLink from a drone internship, which meant MAVROS would have been quicker to set up. However, I chose uXRCE-DDS because it is PX4's official ROS 2 integration path and the direction the ecosystem is moving. It made more sense to use the stack that production systems use rather than the one I was already comfortable with.

Camera Pipeline

Gazebo publishes the simulated camera feed as a Gazebo message internally. To get it into ROS 2, the ros_gz_bridge package was used as a translator between the two. Once bridged, a custom ROS 2 Python node subscribed to the camera topic, converted each frame using cv_bridge, and ran a simple OpenCV-based detector on it.

Perception Step

The perception node uses brightness thresholding to detect bright regions in the camera frame. Each frame is converted to grayscale, thresholded at pixel intensity 200, and contours are extracted. Any contour with an area above 500 pixels is treated as a detected target and gets a bounding box drawn on it. The processed frame is published on a separate ROS 2 topic and also saved locally for inspection. This approach was chosen for simplicity and reliability.

3. Issues and How They Were Resolved

Several blockers came up during setup. Each one is listed with what was tried and how it was fixed -

- **BitLocker blocking dual boot.** Windows BitLocker encrypts the disk and prevents repartitioning. Had to boot into Windows first, disable BitLocker, wait for decryption to complete, then proceed with the Ubuntu install. This alone took most of a day.
- **MicroXRCEAgent shared library not found.** After building and installing the agent, running it gave a shared library error. The library was installed but the system linker cache had not been updated. Running sudo ldconfig fixed it immediately.
- **Arming denied in SITL**. PX4 refused to arm with a health failure error. After checking commander status, the issue turned out to be a missing GCS connection. PX4 requires

QGroundControl to be running and connected before it will arm. Once QGC was open, takeoff worked on the first try.

- **NumPy version conflict with cv_bridge**. The ROS 2 cv_bridge package was compiled against NumPy 1.x, but the system had NumPy 2.2.6 installed. This caused an import error when running the camera node. Fixed by downgrading NumPy to below version 2.
- **Wrong camera topic name.** Initially bridged a generic /camera topic which had no data flowing. Used gz topic -l to list all active Gazebo topics and found the actual camera path, which is model-instance specific and much longer than expected. Updated the bridge command with the correct path and data started flowing at around 17 fps.

4. Validation

Each part of the pipeline was verified before moving to the next step rather than assuming it worked.

- **PX4 SITL:** Confirmed by visual takeoff and hover in Gazebo window, and by QGroundControl showing armed status and altitude increasing.
- **ROS 2 bridge:** Verified by running ros2 topic list and seeing all /fmu/out/ topics appear, then echoing vehicle local position and watching live numbers change during flight.
- **Camera pipeline:** Verified using ros2 topic hz on the camera topic, which showed a consistent 15 to 17 fps stream. Also confirmed by the camera node printing detection logs to the terminal.
- **Perception node:** Confirmed by saved frames showing green bounding boxes drawn by the detector, and by the /camera/processed topic publishing at around 12 fps.
- **Rosbag:** Inspected using ros2 bag info which showed 5 topics recorded over 53 seconds with 4237 total messages, confirming the full pipeline was captured.

5. Limitations

- The bright target detector produces false positives on uniform bright surfaces. When the drone is looking at the sky or a plain white ground, the entire frame can register as a detection. It works better over a textured surface with a distinct bright object.
- The camera topic path is model-instance specific. If a second drone is spawned, the topic name changes and the bridge command needs to be updated manually.
- Startup requires 5 manually ordered terminals. There is no launch file yet, so the correct sequence has to be followed manually each time.

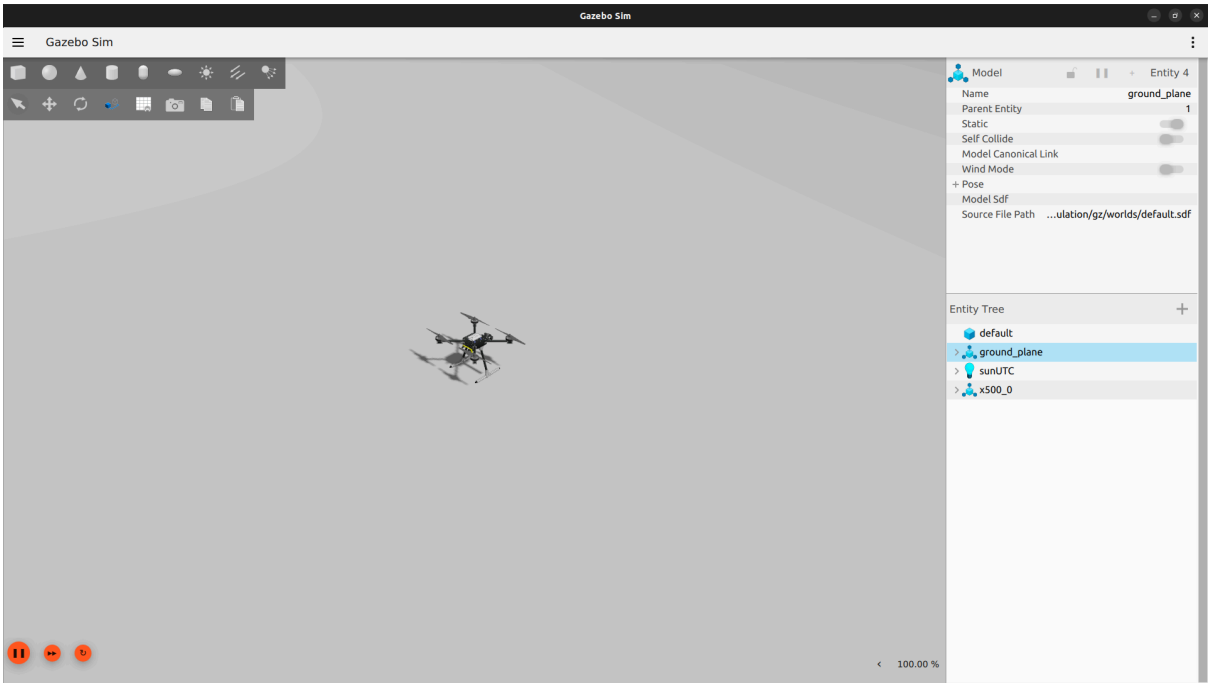
6. Next Steps

Given two more days, the priorities would be:

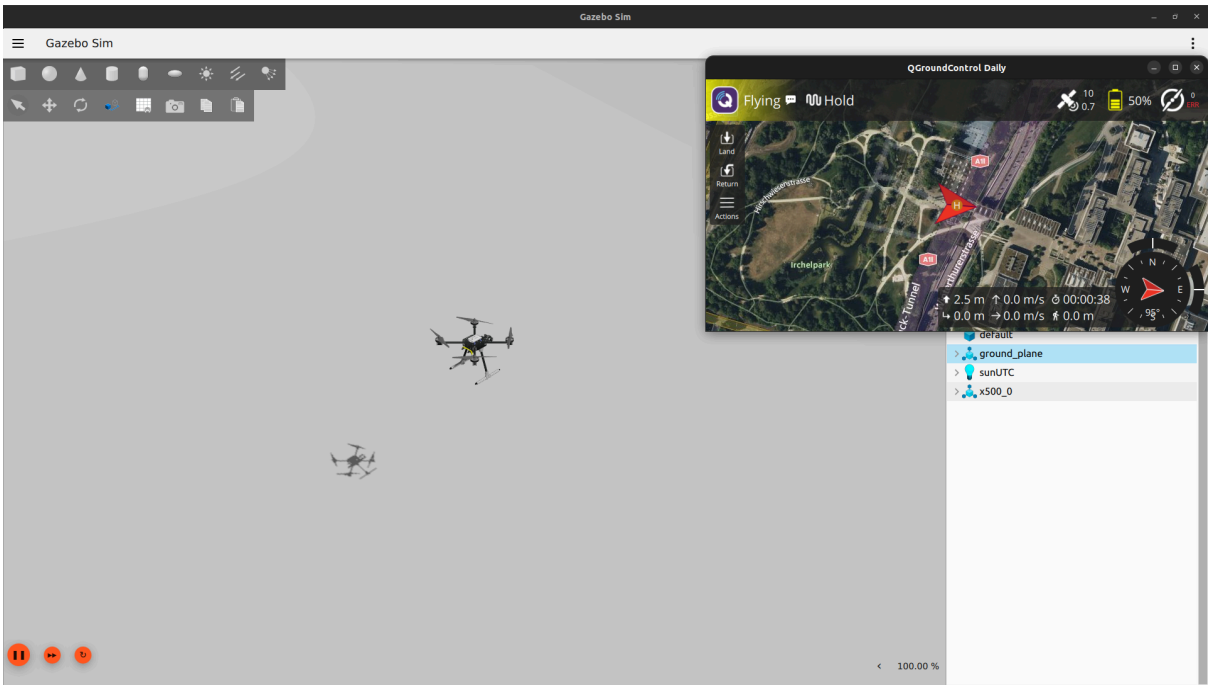
- A single ROS 2 launch file that brings up the bridge, camera node, and any other components in one command with the correct order and dependencies.
- A proper landing pad or ArUco marker detector to replace the brightness threshold approach. This would be more robust and more relevant to the precision landing use case.
- Compressed image transport to reduce rosbag size from several gigabytes to something manageable for longer recording sessions.

Took help from Claude AI and chatGPT for the setup, documentation and fine tuning things, used official guides for tutorials and initial understanding.

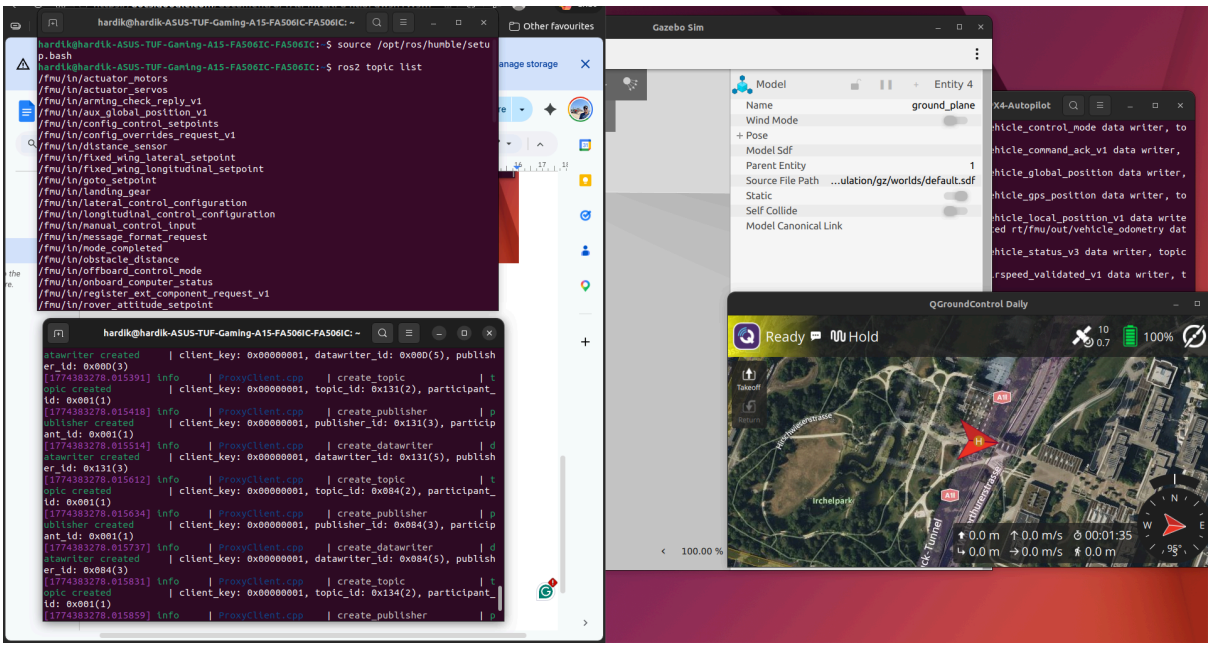
Images



1. **PX4 SITL Working.**



2. **Drone in takeoff mode after GCS Integration**



3. ROS2 Integration - bridge working

```
hardik@hardik-ASUS-TUF-Gaming-A15-FA506IC-FA506IC:~$ ros2 bag info ~/flight_bag

Files:             flight_bag_0.db3
Bag size:          5.0 GiB
Storage id:        sqlite3
Duration:          53.432081012s
Start:             Mar 26 2026 00:30:16.488062021 (1774465216.488062021)
End:               Mar 26 2026 00:31:09.920143033 (1774465269.920143033)
Messages:          4237
Topic information: Topic: /fmu/out/vehicle_status_v3 | Type: px4_msgs/msg/VehicleStatus | Count: 56 | Serialization Format: cdr
                   Topic: /fmu/out/vehicle_attitude | Type: px4_msgs/msg/VehicleAttitude | Count: 1365 | Serialization Format: cdr
                   Topic: /fmu/out/vehicle_local_position_v1 | Type: px4_msgs/msg/VehicleLocalPosition | Count: 1364 | Serialization Format: cdr
                   Topic: /camera/processed | Type: sensor_msgs/msg/Image | Count: 623 | Serialization Format: cdr
                   Topic: /world/default/model/x500_mono_cam_0/link/camera_link/sensor/camera/image | Type: sensor_msgs/msg/Image | Count: 829 | Serialization Format: cdr
```

4. ROS2 Bag Info